

Exercise 1: Singleton Pattern

Python Code:

```
class Logger:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

a = Logger()
b = Logger()
print(a is b)
```

Sample Output Screenshot:

Output
True

Exercise 2: Factory Method Pattern

Python Code:

```
class WordDocument: pass
class PdfDocument: pass

class Factory:
    def create(self, t):
        return WordDocument() if t=='word' else PdfDocument()

print(type(Factory().create('word')).__name__)
```

Sample Output Screenshot:



```
Output
WordDocument
```

Exercise 3: Builder Pattern

Python Code:

```
class Computer:
    def __init__(self,cpu,ram):
        self.cpu=cpu; self.ram=ram

class Builder:
    def cpu(self,c): self._cpu=c; return self
    def ram(self,r): self._ram=r; return self
    def build(self): return Computer(self._cpu,self._ram)

pc=Builder().cpu('i7').ram('16GB').build()
print(pc.cpu, pc.ram)
```

Sample Output Screenshot:

A screenshot of a terminal window showing the output of the Python code. The text "Output" is displayed above the result "i7 16GB".

Output
i7 16GB

Exercise 4: Adapter Pattern

Python Code:

```
class Gateway:
    def pay_gateway(self, amt): print('Paid', amt)

class Adapter:
    def __init__(self, g): self.g=g
    def process_payment(self, amt): self.g.pay_gateway(amt)

Adapter(Gateway()).process_payment(100)
```

Sample Output Screenshot:

Output
Paid 100

Exercise 5: Decorator Pattern

Python Code:

```
class Email:
    def send(self): return 'Email'

class SMS:
    def __init__(self,obj): self.obj=obj
    def send(self): return self.obj.send()+ ' + SMS'

print(SMS(Email()).send())
```

Sample Output Screenshot:

Output
Email + SMS

Exercise 6: Proxy Pattern

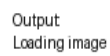
Python Code:

```
class RealImage:
    def display(self): print('Loading image')

class Proxy:
    def __init__(self): self.real=None
    def display(self):
        if not self.real: self.real=RealImage()
        self.real.display()

Proxy().display()
```

Sample Output Screenshot:



```
Output
Loading image
```

Exercise 7: Observer Pattern

Python Code:

```
class Observer:
    def update(self,msg): print(msg)

o=Observer(); o.update('Stock Changed')
```

Sample Output Screenshot:

A screenshot of a terminal window showing the output of the Python code. The text "Output" is on the first line, and "Stock Changed" is on the second line.

Output
Stock Changed

Exercise 8: Strategy Pattern

Python Code:

```
class CreditCard:  
    def pay(self,a): print('Paid',a)  
  
CreditCard().pay(500)
```

Sample Output Screenshot:

A screenshot of a terminal window showing the output of the Python code. The text "Output" is on the first line, and "Paid 500" is on the second line.

Output
Paid 500

Exercise 9: Command Pattern

Python Code:

```
class Light:
    def on(self): print('Light ON')

Light().on()
```

Sample Output Screenshot:



Output
LightON

Exercise 10: MVC Pattern

Python Code:

```
class Student:
    def __init__(self,n): self.name=n

s=Student('John')
print(s.name)
```

Sample Output Screenshot:



```
Output
John
```

Exercise 11: Dependency Injection

Python Code:

```
class Repo:
    def find(self, id): return 'Customer'

class Service:
    def __init__(self, r): self.r=r

print(Service(Repo()).r.find(1))
```

Sample Output Screenshot:



Output
Customer